

# LLM-Based 1-Day Vulnerability Automated Detection & Patch System

Project Proposal

2026.04.26

사이버보안학과 노경민



# Contents

- 1 기술 배경 및 동기
- 2 해결하고자 하는 문제
- 3 문제 해결 질문
- 4 내용 및 방법론
- 5 데이터셋 확보 및 실험 설계
- 6 기존 시스템과의 차별점
- 7 평가 지표 Metric
- 8 수행 계획
- 9 기대 결과물

기술 배경 및 동기

# 왜 1-day 취약점 자동 탐지 및 패치인가

공개 취약점 증가 + 패치 공개 후 공격 전환 속도 증가

40K+

2024년 공개 CVE

공개 취약점 규모가 더 이상 수동 추적 가능한 수준이 아님

Patch → Exploit

1-day 공격 표면

패치 diff는 방어자뿐 아니라 공격자에게도 힌트가 됨

Semantic Gap

기존 도구 한계

컴파일러 옵션, 리팩토링, 그리고 인라이닝 변화에 취약

## 핵심 동기

→ 취약점은 빠르게 공개되지만, 실제 운영 환경의 Binary에서 “이 패치가 내 바이너리에 필요한가?”를 자동으로 판단하는 도구는 부족하다.

해결하고자 하는 문제

# 기존 접근의 두 갈래와 빈틈

구문 기반 매칭은 정확하지만 약하고, 단일 LLM은 유연하지만 믿기 어렵다

## 기존 1-day 탐지 도구

BinXray · FIBER · MVP · PMatch

- 정확한 patch matching
- 결정론적 결과
- 하지만 구문 기반이라 컴파일러·리팩토링 변화에 취약
- patch 의도에 대한 의미적 이해 부족

## 최근 LLM 기반 시도

single-agent classifier · naive prompting

- 의미적 추론과 patch 의도 해석 가능
- 하지만 hallucination에 취약
- 근거 검증 부재로 보안 판단에 직접 사용하기 어려움
- 재현성·설명가능성 부족

**Problem Statement · 의미적 추론과 근거 검증을 동시에 갖춘 1-day binary 취약점 탐지 시스템이 필요하다.**

문제 해결 질문

# 이 프로젝트가 검증하려는 것

C/C++ 바이너리의 Memory Corruption 취약점으로 한정

R Q

Open-source C/C++ binary의 memory corruption 취약점에 대해, multi-agent LLM이 의미적 추론과 근거 검증을 결합해 1-day CVE를 자동 탐지할 수 있는가?

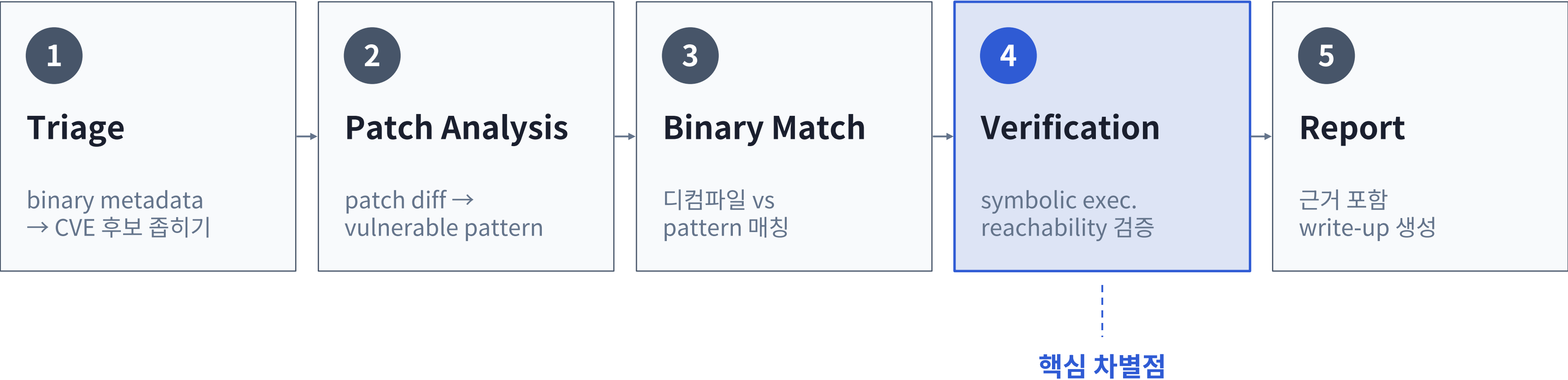
## SCOPE

|         |                          |          |                            |
|---------|--------------------------|----------|----------------------------|
| Target  | Open-source C/C++ binary | CVE      | Heap / Stack overflow, UAF |
| Dataset | Magma                    | Baseline | 단일 LLM (Claude / GPT-4)    |

내용 및 방법론

# Multi-Agent + Verification Layer

C/C++ 바이너리의 Memory Corruption 취약점으로 한정



설 계 원 칙

검증되지 않은 LLM 출력은 reject한다.

## 데이터셋 확보 및 실험 설계

# 무엇을 수집하고 어떻게 검증할 것인가?

Magma를 중심으로 재현 가능한 실험 환경 구성

## 데이터셋 확보

- Magma: ground-truth가 있는 C/C++ 취약점 벤치마크
- 대상: C/C++로 이루어진 open-source 프로젝트
- 각 CVE의 vulnerable/fixed commit, patch diff, PoC input 수집
- 가능하면 Docker 기반 build script로 재현성 확보

## 실험 설계

- 입력: binary + CVE advisory + patch diff
- 출력: 취약 여부, 취약 함수 위치, 근거 문장, confidence
- 비교: single LLM vs multi-agent verifier pipeline
- 실패 케이스: decompiler noise, inlining, false positive 분석

데이터 최소 목표: CVE 15~20개 · 함수 단위 labeling · vulnerable/fixed binary pair · patch diff metadata

기존 연구와의 차별점

# 차별점은 “검증 가능한 LLM의 결과”에 있다

모델 성능은 비슷한데, 실제로 “LLM의 결과가 믿을 만한가?”에 대한 질문은 다른 문제이다

|    |                                |                                               |
|----|--------------------------------|-----------------------------------------------|
| 01 | Semantic Patch Reasoning       | patch diff에서 취약 원인과 수정 의도를 추출                 |
| 02 | Evidence-grounded Verification | binary 근거와 patch 근거가 일치할 때만 accept            |
| 03 | Explainable Write-up           | 탐지 결과를 함수·근거·수정 가이드로 시각화                      |
| 04 | Extensible Framework           | 추후 symbolic execution, ARM cross-arch 분석으로 확장 |

연구 기여 가능성: hallucination을 단순히 줄이는 것이 아니라, 보안 분석 결과를 검증 가능한 artifact로 남기는 방향을 제안한다.

평가 지표 Metric

# 정확도 + 위치 추정 + 설명 품질을 평가한다

탐지 성능뿐만 아니라 보안 도구의 신뢰성도 같이 평가한다

## Detection F1

취약 여부 판별 정확도

binary가 vulnerable인지 fixed인지 분류

## Function Localization

취약 함수 위치 정확도

정답 함수와 agent가 지목한 함수 비교

## Verifier Reject Rate

검증 단계의 보수성

hallucination 의심 결과를 얼마나 걸러내는가

## Write-up Quality

설명 가능성

근거 충실성, 재현 가능성, 수정 가이드 유용성

Baseline 비교: 단일 LLM prompting, rule-based diff matcher, verifier 제거 ablation

수행 계획

# 주차별 개발 및 실험 일정

PoC 완성까지의 일정

|      |               |                                         |
|------|---------------|-----------------------------------------|
| 9주차  | Proposal 확정   | 범위·dataset·평가 지표 고정                     |
| 10주차 | Data Pipeline | Magma build, CVE/patched binary pair 생성 |
| 11주차 | Core Agents   | Patch Analysis + Binary Match agent 구현  |
| 12주차 | Verifier 통합   | Triage/Verifier/Reporter 연결 및 로그 설계     |
| 13주차 | Demo UI       | binary 업로드 → 시각적 report 생성              |
| 14주차 | 평가            | baseline 비교, 실패 케이스 분석                  |
| 15주차 | 최종 정리         | 발표자료·보고서·코드 정리                          |

기대 결과물

# 시각적 Patch Write-Up 제공

라이트업을 결과물로서 제공 + Web Application으로서 동작하도록 구성

vulnscan.app / report

HIGH · CVE-2021-XXXX

parse\_header() — Heap Buffer Overflow

@ 0x401234 · 신뢰도 0.87 · 검증 PASS

- memcpy(buf, input, input\_len);

+ if (input\_len > sizeof(buf)) return -1;

+ memcpy(buf, input, input\_len);

- 1

왜 취약한가

취약 함수의 동작과 공격자 입력 시나리오
- 2

Patch가 한 일

기존 코드 vs patched 코드 시각적 비교
- 3

수정 가이드

어디를 어떻게 고쳐야 하는지 코드 위치까지
- 4

검증 근거

각 agent 판단 + cross-check 통과 여부